

UNIVERSIDAD DE HUELVA

MANTENIMIENTO Y GESTIÓN DEL CAMBIO EN SISTEMAS
SOFTWARE

Memoria Técnica y Arquitectónica

Track 3: Sistema de Gestión de Solicitudes de Soporte Técnico

Autores:

Agustín Rodríguez Aguilar
Sergio Núñez Sierra

Versión del Documento: 1.0 (Relativo a la Release v1.1.1)
30 de mayo de 2026

Índice

1. Introducción y Contexto del Proyecto	2
1.1. Objetivos y Requisitos No Funcionales	2
2. Análisis Funcional y Casos de Uso	3
2.1. Flujos Principales de Negocio	3
3. Decisiones de Arquitectura y Patrones de Diseño	4
3.1. La Regla de Dependencia y el Dominio Puro	4
3.2. Mapeo de Datos y Prevención de Asignación Masiva	4
3.3. El Patrón del Histórico Inmutable	4
3.4. Generación de Identidad Delegada	5
4. Estrategia de Aseguramiento de Calidad (Testing)	6
4.1. Pruebas Unitarias de Dominio	6
4.2. Pruebas de Servicios (Orquestación)	6
4.3. Pruebas de Integración (Persistencia y REST)	6
5. Gestión de Deuda Técnica y Refactorizaciones	7
6. Comparativa de Métricas (SonarCloud)	8
7. DevOps, Integración y Entrega Continua (CI/CD)	9
7.1. Pipeline de Integración Continua (CI)	9
7.2. Release Management (Pipeline de Entrega)	9
8. Manual Técnico de Despliegue y Operación	10
8.1. Arquitectura de Contenedores	10
8.2. Manejo de Condiciones de Carrera (Race Conditions)	10
8.3. Procedimiento de Instalación desde Release Oficial	10
8.4. Verificación y Consumo de la API	11
9. Conclusiones y Líneas de Trabajo Futuro	12

1 Introducción y Contexto del Proyecto

El presente documento constituye la memoria técnica y arquitectónica del proyecto **MGCSS Track 3**, un sistema de *backend* corporativo desarrollado en Java 17 y Spring Boot, orientado a la automatización y gestión del ciclo de vida de incidencias y soporte técnico.

1.1 Objetivos y Requisitos No Funcionales

El desarrollo del sistema no se ha centrado exclusivamente en satisfacer la funcionalidad básica, sino en construir una solución que responda a los más altos estándares de la ingeniería de software moderna. Los objetivos arquitectónicos principales han sido:

- **Alta Mantenibilidad:** Garantizar que el código sea resistente a futuros cambios de requisitos mediante la aplicación de principios SOLID y *Clean Code*.
- **Independencia Tecnológica:** Desacoplar la lógica de negocio de los frameworks (Spring, Hibernate) y de la base de datos (PostgreSQL), asegurando que el dominio central de la aplicación permanezca inmutable ante cambios de infraestructura.
- **Calidad Demostrable:** Establecer un *Quality Gate* restrictivo que impida la integración de código con deuda técnica, *code smells* o vulnerabilidades.
- **Trazabilidad DevOps:** Automatizar todo el ciclo de entrega mediante pipelines de Integración y Entrega Continua (CI/CD), asegurando despliegues reproducibles.

2 Análisis Funcional y Casos de Uso

El dominio del sistema expone una API REST estructurada que permite a los clientes interactuar con la plataforma y a los técnicos gestionar sus cargas de trabajo. El ciclo de vida de una solicitud está gobernado por una máquina de estados estricta.

2.1 Flujos Principales de Negocio

A continuación, se detallan los casos de uso principales implementados, respaldados por la documentación interna del proyecto (`docs/use-cases.md`):

1. Gestión de Solicitudes (Creación y Ciclo de Vida):

- Los clientes pueden registrar nuevas solicitudes de soporte, indicando el problema. El sistema asigna automáticamente el estado **ABIERTA** y sella la fecha de creación.
- Una solicitud no puede transitar de **ABIERTA** a **CERRADA** sin pasar antes por un estado intermedio de gestión.

2. Asignación de Técnicos Cualificados:

- El sistema permite la gestión de personal técnico. Para que un técnico pueda ser asignado a una solicitud, debe poseer una especialidad definida y estar en estado **ACTIVO**.
- Al asignar un técnico a una solicitud **ABIERTA**, el sistema la transita automáticamente al estado **EN_PROCESO**.

3. Resolución y Reapertura:

- El cierre de una solicitud estampa de forma automática la **fecha de cierre**.
- Si el problema persiste, el cliente o administrador puede reabrir la solicitud. El sistema está diseñado para purgar (establecer a **null**) la fecha de cierre anterior, garantizando la consistencia temporal.

4. Trazabilidad Temporal (Novedad de la versión 1.1.0):

- Se ha implementado un sistema de auditoría pasiva. Todo salto de estado en cualquier solicitud genera un registro en el **Histórico de Estados**. Este registro es de solo lectura y permite consultar el ciclo de vida exacto, segundo a segundo, de cualquier incidencia.

3 Decisiones de Arquitectura y Patrones de Diseño

Para satisfacer las premisas de mantenibilidad y escalabilidad, se ha implementado estrictamente el patrón de **Arquitectura Hexagonal (Puertos y Adaptadores)**.

3.1 La Regla de Dependencia y el Dominio Puro

El código fuente se divide en tres capas concéntricas, donde la dependencia de código siempre apunta hacia el centro.

El paquete `com.mgcss.domain` alberga la lógica pura. Las entidades (`Solicitud`, `Tecnico`, `Cliente`) son clases de Java simples (POJOs), sin anotaciones de Spring (`@Entity`, `@Table`) ni dependencias externas. Todas las reglas críticas —como verificar si un técnico está activo antes de asignarlo— se encapsulan en los métodos de estos modelos, garantizando un modelo de dominio rico (Rich Domain Model) en contraposición a un modelo anémico.

3.2 Mapeo de Datos y Prevención de Asignación Masiva

Para evitar vulnerabilidades de seguridad como la asignación masiva (*Mass Assignment*), el sistema implementa el patrón **Data Transfer Object (DTO)**. La API REST nunca recibe ni devuelve entidades de dominio o base de datos. En su lugar, se utilizan características modernas de Java 17, como los `Records`, para crear DTOs inmutables (`SolicitudCreateRequestDto`, `SolicitudResponseDto`).

3.3 El Patrón del Histórico Inmutable

La implementación de la trazabilidad supuso un reto de diseño arquitectónico. Se decidió evitar el uso de *Triggers* de base de datos o interceptores de Hibernate para mantener el control dentro de la capa de aplicación.

En la capa de Dominio, el historial se define como un `Record` inmutable:

```
public record EstadoChange(Estado estado, Date fecha) {}

public void setEstado(Estado nuevoEstado) {
    this.estado = nuevoEstado;
    this.historico.add(new EstadoChange(nuevoEstado, new Date()));
}
```

Listing 1: Generación del Histórico en el Dominio

En la capa de Infraestructura, el `SolicitudRepositoryAdapter` traduce este *Record* a una entidad JPA débil (`EstadoChangeEntity`) vinculada mediante `@OneToMany` con borrado huérfano (`orphanRemoval=true`). Esto permite que el ORM gestione la cascada de datos de forma autónoma.

3.4 Generación de Identidad Delegada

La generación de las claves primarias (IDs) se ha delegado a PostgreSQL utilizando la estrategia `GenerationType.IDENTITY`. A nivel de tabla, esto se traduce en campos `SERIAL` o `BIGSERIAL`. Esta decisión evita cuellos de botella en la aplicación durante la creación concurrente de entidades y previene la fragmentación de la memoria en la capa del ORM.

4 Estrategia de Aseguramiento de Calidad (Testing)

El sistema cuenta con una extensa pirámide de pruebas (Test Pyramid) que valida el comportamiento del software desde la lógica más atómica hasta los puntos de entrada HTTP, utilizando el ecosistema de **JUnit 5**.

4.1 Pruebas Unitarias de Dominio

Ubicadas en `com.mgcss.domain.tests`, estas pruebas son extremadamente rápidas porque no levantan ningún contexto. Verifican estrictamente las reglas de negocio, como `NoActivarTecnicoSinEspecialidadTest` o `NoCerrarSolicitudSiNoEnProcesoTest`.

4.2 Pruebas de Servicios (Orquestación)

En `com.mgcss.service.test`, se emplea **Mockito** (`@Mock`, `@InjectMocks`) para aislar la lógica de los servicios (`SolicitudService`). Se inyectan repositorios falsos para probar escenarios de fallo (ej. intentar buscar un cliente que no existe) sin necesidad de tocar una base de datos real.

4.3 Pruebas de Integración (Persistencia y REST)

Para garantizar que la infraestructura funciona correctamente:

- **Integración JPA:** Anotadas con `@DataJpaTest`, despliegan una base de datos **H2 en memoria** autoconfigurada. Se verifica que Hibernate realiza correctamente los mapeos relacionales, las uniones de tablas (JOINS) y las transacciones de guardado (`JpaSolicitudRepositoryTest`).
- **Pruebas de Controladores:** Mediante `@WebMvcTest` y `MockMvc`, se simulan peticiones HTTP a la capa REST para confirmar que los controladores serializan correctamente el JSON de entrada, delegan a los servicios y responden con los códigos HTTP adecuados (200 OK, 201 Created, 400 Bad Request, 404 Not Found).

5 Gestión de Deuda Técnica y Refactorizaciones

Durante las iteraciones de desarrollo, se realizó un proceso activo de identificación y mitigación de la deuda técnica, documentado formalmente en el archivo `docs/refactor-notes.md`.

Entre las operaciones de limpieza de código (*Clean Code*) destacan:

1. **Estandarización de Paquetes:** Se renombraron los paquetes (de `com.mgcss.track_3` a formato estándar en minúsculas), resolviendo potenciales incidencias de importación en sistemas operativos Case-Sensitive (Linux/Mac).
2. **Modernización de Java:** Se refactorizaron funciones anónimas (*lambdas*) verbosas por **Referencias a Métodos** de Java 8+ (ej. `() -> new IllegalArgumentException()` por `IllegalArgumentException::new`), mejorando significativamente la legibilidad de la recuperación de Optionals.
3. **Limpieza con Lombok:** Se purgaron decenas de líneas de código *boilerplate* correspondientes a constructores vacíos en las entidades JPA, centralizando su generación en la anotación `@NoArgsConstructor`.
4. **Optimización de JUnit 5:** Se aplicó la convención moderna de pruebas eliminando los modificadores de acceso `public` de todas las clases y métodos de test, ya que la versión 5 del framework no los requiere gracias a su motor reflectivo subyacente.

6 Comparativa de Métricas (SonarCloud)

La calidad del código ha sido monitorizada y validada mediante análisis estático continuo en **SonarCloud**, acoplado directamente al ciclo de integración.

El estado final del proyecto demuestra el éxito de las refactorizaciones aplicadas. A continuación, se presenta la comparativa evolutiva desde los análisis iniciales hasta la *Release v1.1.1* candidata a auditoría:

Métrica Auditable (SonarCloud)	Estado Inicial (Auditoría 1)	Release Final (v1.1.1)
Bugs de Software	Presencia detectada	0
Vulnerabilidades Críticas	Presencia detectada	0
Code Smells	Abundantes (<i>Boilerplate</i>)	0
Security Hotspots	1 (Bloqueante por revisión)	100 % Revisados (Safe)
Deuda Técnica Total (Días)	>2 Días	0 Minutos
Calidad Global (Quality Gate)	Failed (Fallido)	Passed (Aprobado)

Tabla 1: Evolución y resolución de deuda técnica durante el proyecto.

Nota sobre los *Security Hotspots*: El punto caliente de seguridad reportado se debió a la detección del uso de credenciales en el archivo YAML de GitHub Actions (`GITHUB_TOKEN`). Este comportamiento fue auditado manualmente por el equipo de desarrollo y marcado como seguro y legítimo, desbloqueando el paso a producción.

7 DevOps, Integración y Entrega Continua (CI/CD)

El ciclo de vida de la aplicación se encuentra totalmente automatizado mediante flujos de trabajo en **GitHub Actions**, garantizando entregas ágiles, auditables y reproducibles.

7.1 Pipeline de Integración Continua (CI)

Cualquier subida de código (*Push*) a ramas de funcionalidad o *Pull Requests* hacia la rama **main** activa el archivo `.github/workflows/ci.yml`. Este proceso provisiona una máquina Ubuntu, instala el JDK 17 y ejecuta Maven para compilar el proyecto. A continuación, lanza la suite completa de pruebas unitarias y de integración. Finalmente, recolecta el reporte de cobertura de JaCoCo y lo envía a SonarCloud. **Si el Quality Gate falla, la Pull Request queda bloqueada, impidiendo que el código defectuoso llegue a producción.**

7.2 Release Management (Pipeline de Entrega)

Para la gestión de versiones, se adoptó el estándar de Semantic Versioning (SemVer) bajo el formato **MAJOR.MINOR.PATCH**. La publicación de una nueva versión está automatizada por el flujo `release.yml`, el cual se dispara exclusivamente ante la detección de un *Tag* etiquetado con la letra "v" (ej. `v1.1.1`).

El flujo de liberación ejecuta las siguientes fases:

1. **Quality Gate Restrictivo:** Re-evaluación del código para garantizar que la versión final es impecable.
2. **Construcción Docker:** Generación de la imagen del contenedor (`mgcss-track`) utilizando el `Dockerfile` alojado en la raíz del proyecto.
3. **Empaquetado de Artefacto:** Compilación del binario ejecutable `.jar`.
4. **Publicación en GitHub:** Creación autónoma de la *Release* en el repositorio. El sistema genera automáticamente las notas de la versión (*Release Notes*) basándose en el historial de PRs y adjunta el ejecutable `.jar` como un *Asset* descargable para el usuario final.

8 Manual Técnico de Despliegue y Operación

El sistema ha sido *containerizado* bajo la tecnología Docker. Esto permite su despliegue en cualquier entorno (desarrollo, pruebas, producción) sin requerir configuración compleja de la máquina anfitriona.

8.1 Arquitectura de Contenedores

El despliegue se orquesta mediante `docker-compose.yml`, el cual define dos servicios interdependientes:

- **postgres:** Contenedor basado en `postgres:16-alpine`. Expone el puerto 5432 y monta un volumen (`postgres_data`) para garantizar la persistencia de datos ante reinicios.
- **app:** Contenedor de la aplicación Spring Boot, que inyecta por variables de entorno la cadena de conexión JDBC hacia la base de datos.

8.2 Manejo de Condiciones de Carrera (Race Conditions)

Uno de los problemas clásicos del despliegue concurrente es que el *backend* intente arrancar antes de que la base de datos esté lista para recibir conexiones, provocando un error fatal. Para solucionarlo de forma elegante, el entorno implementa un **Healthcheck** nativo:

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U myuser -d mydatabase"]
  interval: 5s
  timeout: 5s
  retries: 5
```

Listing 2: Comprobación de salud en PostgreSQL

El contenedor de la aplicación incluye la directiva `depends_on: condition: service_healthy`, lo que obliga a Docker a pausar el arranque de Spring Boot hasta que el test `pg_isready` confirme que la base de datos es totalmente funcional.

8.3 Procedimiento de Instalación desde Release Oficial

1. Acceda a la pestaña **Releases** del repositorio en GitHub.
2. En la versión deseada (ej. `v1.1.1`), descargue el archivo `compose.yaml` y el binario `*.jar` adjunto en la sección de *Assets*.
3. Sitúe ambos archivos en un directorio local vacío.
4. Ejecute el comando de orquestación en segundo plano:

```
docker compose up --build -d
```

8.4 Verificación y Consumo de la API

Una vez que ambos contenedores se muestren como saludables (*healthy*), la API expone una interfaz visual interactiva generada dinámicamente con Swagger/OpenAPI. Puede acceder a ella en cualquier navegador a través de:

<http://localhost:8080/swagger-ui/index.html>

9 Conclusiones y Líneas de Trabajo Futuro

La ejecución del Track 3 ha culminado en la entrega de un producto software que cumple rigurosamente con los paradigmas actuales de la ingeniería. La separación de responsabilidades mediante la Arquitectura Hexagonal y la fuerte automatización a través del ecosistema de GitHub Actions proporcionan un suelo extremadamente fértil para el crecimiento de la plataforma.

Como parte del ciclo de mejora continua, se identifican las siguientes líneas de trabajo para la siguiente evolución del sistema:

- **Implementación de Seguridad (Autenticación/Autorización):** Incorporación de *Spring Security* junto con la generación y validación de tokens JWT (JSON Web Tokens). Esto permitirá diferenciar roles entre "Cliente", "Técnicos" y "Administrador", restringiendo el acceso a los *endpoints*.
- **Arquitectura Orientada a Eventos (EDA):** Desacoplar el envío de notificaciones del flujo principal. Se propone integrar un *Message Broker* (como RabbitMQ o Apache Kafka) para publicar eventos del tipo `SolicitudEstadoCambiadoEvent`, que posteriormente serían consumidos por un microservicio encargado del envío asíncrono de correos electrónicos.
- **Capa de Presentación (Frontend):** Desarrollar una interfaz gráfica de usuario en formato SPA (Single Page Application) utilizando frameworks modernos como React o Angular, que consuma directamente los *endpoints* expuestos por la API REST actual.
- **Despliegue en Cloud:** Extender las capacidades de `release.yml` para orquestar un despliegue continuo hacia plataformas en la nube (AWS ECS, Google Cloud Run o Azure App Services), cerrando el círculo completo de *Continuous Deployment*.